BTS FILIÈRE : DAI

Les Design Patterns

Solutions réutilisables aux problèmes de conception logicielle

1. Qu'est-ce qu'un Design Pattern ?

Un **Design Pattern** (Patron de Conception) est une solution standardisée et éprouvée à un problème récurrent rencontré lors du développement logiciel.

Avantages :

- Améliore la lisibilité et la maintenance.
- Fournit un langage commun aux développeurs.
- Réduit les bugs architecturaux.

> *"C'est un plan d'architecte réutilisable que l'on adapte à notre contexte spécifique."*

2. Classification des Patterns

Création

Gèrent la manière dont les objets sont instanciés (créés).

Ex: Singleton, Factory.

Structure

Gèrent l'organisation et l'assemblage des classes.

Ex: Adapter, Facade.

Comportement

Gèrent la communication entre les objets.

Ex: Observer, Strategy.

3. Le Pattern Singleton

Objectif : S'assurer qu'une classe n'a qu'une seule et unique instance dans toute l'application et fournir un point d'accès global.

Cas d'usage :

- Gestionnaire de base de données.
- Stockage de session utilisateur.
- Journalisation (Logger).

Structure :

1. Constructeur private.
2. Instance statique de la classe.
3. Méthode statique getInstance().

Singleton : Code Java

```java
public class SessionManager {
    private static SessionManager instance;
    private String userToken;

    // Constructeur privé;
    private SessionManager() { };

    public static SessionManager getInstance() {
        if (instance == null) {
            instance = new SessionManager();
        }
        return instance;
    }

    public void setToken(String token) {
        this.userToken = token;
    }
}
```

4. Le Pattern Factory (Fabrique)

Objectif : Définir une interface pour créer un objet, mais laisser les sous-classes décider de la classe à instancier.

On délègue la logique de création à une méthode spécialisée au lieu d'utiliser new partout.

Permet de découpler le code appelant de l'implémentation concrète.

Factory : Code Java

```java
public class NotificationFactory {
    public Notification create(String type) {
        if (type.equals("SMS")) {
            return new SmsNotification();
        } else if (type.equals("EMAIL")) {
            return new EmailNotification();
        }
        return null;
    }
}
// Utilisation;
Notification n = factory.create("SMS");
n.send();
```

5. Le Pattern Observer

Objectif : Définir une dépendance de un à plusieurs. Quand un objet change d'état, tous ses abonnés sont prévenus automatiquement.
Fondement de LiveData : En Android, la Vue observe les données dans le ViewModel.

Idéal pour les notifications système et les mises à jour d'interface réactives.

Observer : Logique Java

```java
// Définition de l'interface;
public interface Observer {
    void update(String message);
}

public class Subject {
    private List list = new ArrayList<>();

    public void notifyAll(String msg) {
        for (Observer obs : list) {
            obs.update(msg);
        }
    }
}
```

6. Le Pattern Adapter

Objectif : Permettre à des classes aux interfaces incompatibles de travailler ensemble.

Application Android : Le RecyclerView.Adapter adapte vos données (List) en Vues (ViewHolder).

Il agit comme un convertisseur de prise électrique universel.

7. Le Pattern Facade

Objectif : Fournir une interface simplifiée à un système complexe (bibliothèque, ensemble de classes).

Exemple : Une classe FileManager qui cache la complexité des FileOutputStream et BufferedWriter.

```
// Classe Facade simplifiée;
public class MyFileManager {
    public void save(String txt) {
        // Cache toute la complexité Java IO;
        FileUtils.write(txt);
    }
}
```

8. Architecture : Logique vs UI

Le plus gros défi d'un développeur DAI est la **Séparation des Responsabilités (SoC)**.

Logique (Logic)

Calculs, accès aux données, gestion des flux I/O.

Classes : FileManager, Repository.

Interface (UI)

Affichage, clics utilisateur, navigation.

Classes : Activity, Fragment.

9. Les Principes S.O.L.I.D

5 règles pour un code de qualité professionnelle

:

Single Responsibility	Une classe = Une seule tâche.
Open/Closed	Ouvert à l'extension, fermé à la modification.
Liskov Substitution	Les sous-classes doivent être substituables.
Interface Segregation	Préférer plusieurs interfaces spécifiques.
Dependency Inversion	Dépendre des abstractions, pas des détails.

10. Pattern vs Architecture

Design Pattern

Solution locale à un problème précis dans une classe.

Ex: Singleton.

Architecture

Organisation globale du projet (structure des dossiers et flux).

Ex: MVC, MVVM.

11. Le Pattern Repository

Indispensable pour vos projets DAI. Il centralise l'accès aux données.

```
public class ProductRepository {
    public List getAll() {
        // Décide de lire le Fichier CSV ou SQLite;
        return FileManager.read();
    }
}
```

L'Activity appelle le Repository, elle ne sait pas comment les données sont stockées.

12. Comparaison des Stockages

Solution	Structure	Usage	Vitesse
Fichiers	Simple / CSV	Données brutes	Moyenne
SQLite	Tabulaire	Relations complexes	Élevée
Firebase	JSON / Cloud	Synchronisation	Dépend réseau

13. Les Anti-Patterns (À éviter)

God Object
Une classe qui fait tout (ex: Activity de 2000 lignes qui gère le SQL, le réseau et l'UI).

Hard Coding
Écrire des valeurs en dur au lieu d'utiliser des constantes ou des fichiers ressources.

14. L'Injection de Dépendance

Ne pas créer les dépendances à l'intérieur d'une classe, mais les "passer" (injecter).

```
// Mauvais couplage;
this.manager = new FileManager();

// Bon couplage (Injection via constructeur);
public MyRepo(FileManager manager) {
    this.manager = manager;
}
```

15. Compétences attendues

- Identifier le bon pattern pour un problème technique.
- Implémenter un Singleton thread-safe.
- Séparer le code métier du code d'interface.
- Expliquer l'intérêt des principes SOLID.
- Utiliser le pattern Facade pour simplifier les accès I/O.

Conclusion

Les Design Patterns ne sont pas des règles rigides, mais des outils pour écrire du code **propre, testable et évolutif**.

Des questions ?